



UNIVERSIDAD TECNOLÓGICA DE TEHUACÁN

INGENIERÍA EN DESARROLLO Y GESTION DE SOFTWARE

ASIGNATURA: DESARROLLO WEB PROFESIONAL

8º CUATRIMESTRE GRUPO “A”

ARQUITECTURA DE AUTENTICACIÓN Y CONTROL DE ROLES

PROFR. JOSE MIGUEL CARRERA PACHECHO

LILIA HERNÁNDEZ TUN

¿Qué debes investigar?

¿CÓMO DEBE COMPORTARSE UN FORMULARIO ACCESIBLE?

Un formulario accesible debe seguir los principios **POUR** (Perceptible, Operable, Comprensible y Robusto) y comportarse de la siguiente manera:

Etiquetado Correcto y Explícito: La base de la accesibilidad es que cada control tenga una función clara.

- **Asociación label + input:** Se debe usar el atributo `for` en la etiqueta `<label>` apuntando al `id` del `<input>`. Esto permite que los lectores de pantalla anuncien el nombre del campo al recibir el foco y amplía el área de clic para personas con problemas de motricidad fina.
- **Agrupación lógica:** Para grupos de botones de radio o checkboxes, se debe usar `<fieldset>` para agruparlos y `<legend>` para darles un título general.
- **Evitar el Placeholder como etiqueta:** El texto de ejemplo (placeholder) no sustituye a la etiqueta, ya que desaparece al escribir y no es leído por todos los lectores de pantalla de forma consistente.

2. Navegación y Operabilidad por Teclado

- **Orden de tabulación lógico:** El foco debe seguir el orden visual de los campos (usualmente de arriba hacia abajo y de izquierda a derecha). Si el orden es confuso, se puede ajustar con el atributo `tabindex`, aunque lo ideal es que el HTML sea semántico y natural.
- **Indicador de foco visible:** Nunca se debe ocultar el contorno (outline) de los campos. El usuario que navega con teclado debe saber exactamente dónde está posicionado.
- **Acción de envío:** El formulario debe poder enviarse presionando la tecla **Enter** desde cualquier campo de texto, lo cual se logra envolviendo los campos en una etiqueta `<form>` y teniendo un `<button type="submit">`.

3. Validaciones y Mensajes de Error (Feedback): El formulario debe "comunicar" su estado en todo momento, especialmente cuando algo sale mal.

- **Validación en vivo y programática:** Se debe indicar qué campos son obligatorios no solo con un asterisco (*), sino de forma programática usando el atributo required.
- **Atributos ARIA para errores:**
 - aria-invalid="true": Se activa en el input cuando hay un error para avisar al usuario.
 - aria-describedby: Vincula el input con el ID del texto de error, para que el lector de pantalla lea el mensaje de error automáticamente al entrar al campo.
- **Sugerencias de corrección:** Los mensajes no deben ser genéricos (como "Error"). Deben ser claros: "Introduce un correo válido (ejemplo@correo.com)".
- **Alertas inmediatas:** Los errores deben aparecer en contenedores con role="alert" o aria-live="polite" para que sean anunciados por voz en cuanto ocurran.

¿QUÉ ERRORES COMUNES EXISTEN EN FORMULARIOS DE LOGIN?

1. Errores de Diseño y Usabilidad (UX)

- **Placeholder como etiqueta:** Un error crítico es usar el placeholder para decir "Usuario" o "Contraseña".
- **Ocultar el indicador de foco:** Muchos desarrolladores quitan el recuadro que rodea al campo activo (con outline: none en CSS). Esto impide que una persona que usa teclado sepa dónde está parada.
- **Falta de opción para "Mostrar Contraseña":** Obligar al usuario a escribir a ciegas aumenta la tasa de error.

- **Bloqueo del pegado (Paste):** Evitar que el usuario pegue su contraseña desde un gestor de contraseñas (como 1Password o Chrome) reduce la seguridad, ya que empuja al usuario a usar claves cortas y fáciles de recordar.

2. Errores de Accesibilidad (A11y)

- **Validaciones solo por color:** Mostrar un error poniendo el borde del input en rojo sin un texto descriptivo. Las personas con daltonismo no percibirán el error.
- **Instrucciones inaccesibles:** Si hay requisitos de contraseña (ej. "mínimo 8 caracteres"), a menudo solo aparecen tras fallar. La solución es poner la instrucción visible desde el inicio y conectarla con aria-describedby.
- **Timeouts de sesión silenciosos:** En formularios largos o logins con MFA (autenticación de dos pasos), la sesión puede expirar sin avisar al usuario, provocando que pierda los datos ingresados al dar clic en "Enviar".

3. Errores de Comunicación y Feedback

- **Mensajes de error ambiguos o técnicos:** Mostrar códigos de error de base de datos o mensajes como "Error 401" en lugar de algo comprensible para el usuario.
- **Falta de "Autofocus" inteligente:** No colocar el foco automáticamente en el primer campo vacío al cargar la página obliga al usuario a realizar un clic o tabulación extra innecesaria.
- **No limpiar campos sensibles:** Tras un intento fallido, a veces el formulario mantiene la contraseña anterior en texto plano o no limpia correctamente los estados de error anteriores.

4. Errores Críticos de Seguridad en el Login

- **Enumeración de usuarios:** El error más grave, es decir: *"El usuario no existe"* o *"Contraseña incorrecta"*. Esto permite a un atacante saber qué correos electrónicos están registrados en tu base de datos.
 - **Solución:** Usar siempre un mensaje genérico: *"Credenciales incorrectas"* o *"Usuario o contraseña no válidos"*.
- **No gestionar el envío múltiple:** No deshabilitar el botón de "Entrar" después del primer clic, lo que puede causar múltiples peticiones al servidor si el usuario presiona varias veces por impaciencia.

¿CÓMO SE MANEJAN SESIONES DE FORMA SEGURA?

1. El uso de Cookies Seguras

Si utilizas cookies para almacenar el ID de sesión (o un token JWT), estas deben configurarse con tres "banderas" (flags) obligatorias:

- **HttpOnly:** Evita que el token sea accesible mediante scripts de JavaScript. Esto protege al usuario de ataques **XSS (Cross-Site Scripting)**, donde un atacante intenta robar la sesión con un script malicioso.
- **Secure:** Asegura que la cookie solo se envíe a través de conexiones cifradas **HTTPS**. Nunca viajará en texto plano por HTTP.
- **SameSite (Strict o Lax):** Evita ataques de **CSRF (Cross-Site Request Forgery)**, impidiendo que el navegador envíe la cookie de sesión si la petición viene de un sitio externo.

2. Generación y Almacenamiento

- **Identificadores Complejos:** El ID de sesión debe ser una cadena larga, aleatoria y criptográficamente segura. No debe ser predecible (como un ID incremental 1, 2, 3...).
- **No guardar datos sensibles en el cliente:** El navegador solo debe guardar el identificador (token). Los datos reales del usuario (email, privilegios, saldo)

deben residir en la base de datos o en una caché del servidor (como Redis), vinculados a ese ID.

3. Ciclo de Vida de la Sesión

Una sesión segura no es eterna. Debes implementar:

- **Expiración por Inactividad:** Si el usuario no interactúa en X minutos, la sesión debe invalidarse.
- **Expiración Absoluta:** Forzar un nuevo login después de cierto tiempo (ej. 24 horas), incluso si el usuario ha estado activo.
- **Invalidación al cerrar sesión (Logout):** El botón de "Cerrar sesión" debe borrar la cookie en el navegador y **destruir** la sesión en el servidor simultáneamente.

4. Protección contra el Secuestro de Sesión

- **Regeneración de ID:** Es vital cambiar el ID de sesión inmediatamente después de que el usuario se loguee. Esto evita el ataque de "Fijación de Sesión", donde un atacante le da un ID válido al usuario antes de que entre.
- **Vinculación al contexto:** Algunos sistemas verifican que la dirección IP o el *User-Agent* (navegador) no cambien drásticamente durante la sesión, aunque esto último puede dar problemas en conexiones móviles.

¿CÓMO SE MUESTRAN ERRORES DE AUTENTICACIÓN SIN FILTRAR INFORMACIÓN SENSIBLE?

1. Mensajes Genéricos (Indiferenciación)

El error más común es ser "demasiado específico". Para proteger la privacidad de tus usuarios, debes unificar las respuestas:

- **Mal:** "El correo electrónico no está registrado" o "Contraseña incorrecta para el usuario admin".
- **Bien:** "Usuario o contraseña incorrectos" o "Credenciales no válidas".

- **Por qué:** Si el sistema confirma que un correo existe, un atacante puede recolectar una lista de emails de tu base de datos para ataques dirigidos o *phishing*.

2. Respuestas de Tiempo Constante

Los atacantes avanzados usan **ataques de temporización**. Si el servidor responde en 10ms cuando el usuario no existe, pero tarda 50ms cuando el usuario existe (porque se queda validando la contraseña), el atacante puede deducir la existencia del usuario.

- **Solución:** Asegúrate de que tu código realice una operación de "hash" de contraseña (como bcrypt) incluso si el usuario no es encontrado, para que el tiempo de respuesta sea siempre similar.

3. Manejo de Bloqueos y Multi-factor (MFA)

- **Bloqueo de cuenta:** Si una cuenta se bloquea por demasiados intentos fallidos, el mensaje debe seguir siendo neutral: *"Si la cuenta existe, ha sido bloqueada temporalmente por seguridad. Inténtalo más tarde"*.
- **Flujo de MFA:** Si el usuario pasa el primer paso (contraseña), no digas "Contraseña correcta."

REFERENCIAS

González, M. (s.f.). *Formularios accesibles: etiquetas, validaciones y feedback | Checklist + Snippets*. Blog de Marta González. <https://martagonzalez.net>

Hostragons. (s.f.). *Gestión y seguridad de sesiones de usuario*. Hostragons® - Infraestructura Web. <https://hostragons.com>

Microsoft Learn. (2023, 12 de octubre). *Solución de problemas de escenarios comunes para formularios*. Microsoft Docs. <https://learn.microsoft.com>

Unagi. (s.f.). *Formularios | Guía rápida de accesibilidad web*. Unagi - Accesibilidad Digital. <https://unagi.com.ar>

Web Accessibility Initiative (WAI). (s.f.). *Accesibilidad Web: Conceptos generales sobre la accesibilidad de formularios*. W3C. <https://www.w3.org/WAI/>